

Ejercicios Resueltos

Introducción

Esta sección contiene ejercicios prácticos resueltos que integran todos los conceptos aprendidos: manipulación del DOM, eventos, estilos y JavaScript básico. Los ejercicios están ordenados de menor a mayor dificultad.

Ejercicios básicos

Ejercicio 1: Variables y tipos de datos primitivos

Declara las siguientes variables y muéstralas por consola junto al tipo de dato con el que se corresponde:

- `nombre`, con el valor `"Juan"`
- `edad`, con el valor `25`
- `altura`, con el valor `1.75`
- `esEstudiante`, con el valor `true`
- `ciudad`, sin asignarle ningún valor
- `pais`, con el valor `null`

Además, realiza las siguientes operaciones con las variables numéricas y muestra los resultados:

- Suma `5` a la edad
- Resta `3` a la edad
- Multiplica la altura por `2`
- Divide la edad entre `5`

► Solución

```
// Declarar variables de diferentes tipos
let nombre = "Juan";
let edad = 25;
let altura = 1.75;
let esEstudiante = true;
let ciudad;
```



```
let pais = null;

// Mostrar el valor y tipo de cada variable
console.log("Nombre:", nombre, "- Tipo:", typeof nombre);
console.log("Edad:", edad, "- Tipo:", typeof edad);
console.log("Altura:", altura, "- Tipo:", typeof altura);
console.log("Es estudiante:", esEstudiante, "- Tipo:", typeof esEstudiante);
console.log("Ciudad:", ciudad, "- Tipo:", typeof ciudad);
console.log("País:", pais, "- Tipo:", typeof pais);

// Operaciones con números
let suma = edad + 5;
let resta = edad - 3;
let multiplicacion = altura * 2;
let division = edad / 5;

console.log("Suma:", suma);
console.log("Resta:", resta);
console.log("Multiplicación:", multiplicacion);
console.log("División:", division);
```

Conceptos practicados:

- Declaración de variables con `let`
- Tipos de datos primitivos: string, number, boolean, undefined, null
- Operador `typeof`
- Operaciones aritméticas básicas

Ejercicio 2: Strings y sus métodos

Dada la siguiente cadena de texto: `"Hola Mundo desde JavaScript"`, realiza las siguientes operaciones y muestra los resultados por consola:

1. Mostrar la longitud de la cadena
2. Convertir la cadena a mayúsculas
3. Convertir la cadena a minúsculas
4. Comprobar si incluye la palabra `"Mundo"`
5. Buscar el índice donde empieza `"JavaScript"`
6. Extraer los primeros 4 caracteres usando `substring()`
7. Extraer los caracteres desde la posición 5 hasta la 10 usando `slice()`

8. Reemplazar "Mundo" por "Planeta"
9. Dividir la cadena en un array de palabras
10. Eliminar los espacios en blanco del inicio y final de " JavaScript "
11. Crear un mensaje usando template literals con las variables nombre = "Ana" y edad = 28

► Solución

```
let texto = "Hola Mundo desde JavaScript";

// Longitud de la cadena
console.log("Longitud:", texto.length);

// Convertir a mayúsculas y minúsculas
console.log("Mayúsculas:", texto.toUpperCase());
console.log("Minúsculas:", texto.toLowerCase());

// Buscar subcadenas
console.log("Incluye 'Mundo':", texto.includes("Mundo"));
console.log("Índice de 'JavaScript':", texto.indexOf("JavaScript"));

// Extraer partes de la cadena
console.log("Substring:", texto.substring(0, 4));
console.log("Slice:", texto.slice(5, 10));

// Reemplazar texto
console.log("Reemplazar:", texto.replace("Mundo", "Planeta"));

// Dividir cadena en array
let palabras = texto.split(" ");
console.log("Palabras:", palabras);

// Eliminar espacios
let textoConEspacios = " JavaScript ";
console.log("Trim:", textoConEspacios.trim());

// Template Literals
let nombre = "Ana";
let edad = 28;
let mensaje = `Mi nombre es ${nombre} y tengo ${edad} años`;
console.log(mensaje);
```

Conceptos practicados:

- Métodos de string: `length`, `toUpperCase()`, `toLowerCase()`, `includes()`, `indexOf()`
- `substring()`, `slice()`, `replace()`, `split()`, `trim()`
- Template literals (backticks)

Ejercicio 3: Arrays - Creación y métodos básicos

Crea un array llamado `frutas` con los siguientes elementos: `"manzana"`, `"pera"`, `"naranja"`, `"plátano"`. Realiza las siguientes operaciones:

1. Mostrar por consola el array completo
2. Acceder y mostrar el primer elemento
3. Acceder y mostrar el último elemento
4. Añadir `"uva"` al final del array usando `push()`
5. Añadir `"fresa"` al principio del array usando `unshift()`
6. Eliminar el último elemento usando `pop()` y mostrar el elemento eliminado
7. Eliminar el primer elemento usando `shift()` y mostrar el elemento eliminado
8. Buscar el índice de `"pera"` usando `indexOf()`
9. Comprobar si el array incluye `"manzana"` usando `includes()`
10. Extraer los elementos desde el índice 1 hasta el 3 (sin incluir) usando `slice()`
11. Unir todos los elementos en una cadena separada por comas usando `join()`
12. Crear un array `numerosDesordenados` con los valores `[5, 2, 8, 1, 9]` y ordenarlo de menor a mayor
13. Invertir el orden del array de frutas usando `reverse()`

► Solución

```
// Crear arrays
let numeros = [1, 2, 3, 4, 5];
let frutas = ["manzana", "pera", "naranja", "plátano"];
let mixto = [1, "dos", true, null, { nombre: "Juan" }];

console.log("Array de números:", numeros);
console.log("Array de frutas:", frutas);
console.log("Array mixto:", mixto);

// Acceder a elementos
```



```

console.log("Primera fruta:", frutas[0]);
console.log("Última fruta:", frutas[frutas.length - 1]);

// Añadir elementos
frutas.push("uva"); // Añadir al final
console.log("Después de push:", frutas);

frutas.unshift("fresa"); // Añadir al principio
console.log("Después de unshift:", frutas);

// Eliminar elementos
let ultimaFruta = frutas.pop(); // Eliminar del final
console.log("Fruta eliminada:", ultimaFruta);
console.log("Después de pop:", frutas);

let primeraFruta = frutas.shift(); // Eliminar del principio
console.log("Fruta eliminada:", primeraFruta);
console.log("Después de shift:", frutas);

// Buscar elementos
console.log("Índice de 'pera':", frutas.indexOf("pera"));
console.log("¿Incluye 'manzana'?:", frutas.includes("manzana"));

// Extraer parte del array
let algunasFrutas = frutas.slice(1, 3);
console.log("Slice (1, 3):", algunasFrutas);

// Unir elementos
let frutasTexto = frutas.join(", ");
console.log("Join:", frutasTexto);

// Ordenar
let numerosDesordenados = [5, 2, 8, 1, 9];
console.log("Original:", numerosDesordenados);
console.log("Ordenado:", numerosDesordenados.sort((a, b) => a - b));

// Invertir
console.log("Invertido:", frutas.reverse());

```

Conceptos practicados:

- Creación de arrays
- Acceso a elementos por índice
- Métodos: push(), pop(), shift(), unshift()

- Métodos: `indexOf()`, `includes()`, `slice()`, `join()`
- Métodos: `sort()`, `reverse()`

Ejercicio 4: Objetos literales

Crea un objeto llamado `persona` con las siguientes propiedades:

- `nombre`: "María"
- `apellido`: "García"
- `edad`: 30
- `ciudad`: "Madrid"
- `profesion`: "Desarrolladora"
- `hobbies`: ["leer", "programar", "viajar"]
- `activo`: true

Realiza las siguientes operaciones:

1. Acceder y mostrar el nombre usando notación punto
2. Acceder y mostrar el apellido usando notación de corchetes
3. Modificar la edad a 31
4. Modificar la ciudad a "Barcelona"
5. Añadir una nueva propiedad `email` con el valor "maria@ejemplo.com"
6. Añadir una nueva propiedad `telefono` con el valor "123456789"
7. Eliminar la propiedad `activo` usando `delete`

Además, crea un objeto `calculadora` con los siguientes métodos:

- `sumar(a, b)`: retorna la suma de a y b
- `restar(a, b)`: retorna la resta de a y b
- `multiplicar(a, b)`: retorna la multiplicación de a y b
- `dividir(a, b)`: retorna la división de a entre b (controla el error de división por cero)

Prueba los métodos de la calculadora con diferentes valores.

Por último, crea un objeto `coche` con propiedades `marca`, `modelo` y `año`, y un método `descripcion()` que retorne una cadena descriptiva usando `this`.

► Solución

```
// Crear un objeto
let persona = {
  nombre: "María",
  apellido: "García",
  edad: 30,
  ciudad: "Madrid",
  profesion: "Desarrolladora",
  hobbies: ["leer", "programar", "viajar"],
  activo: true
};

// Acceder a propiedades
console.log("Nombre:", persona.nombre);
console.log("Apellido:", persona["apellido"]);
console.log("Edad:", persona.edad);

// Modificar propiedades
persona.edad = 31;
persona.ciudad = "Barcelona";
console.log("Edad actualizada:", persona.edad);
console.log("Ciudad actualizada:", persona.ciudad);

// Añadir nuevas propiedades
persona.email = "maria@ejemplo.com";
persona.telefono = "123456789";
console.log("Email:", persona.email);

// Eliminar propiedades
delete persona.activo;
console.log("Objeto después de delete:", persona);

// Métodos en objetos
let calculadora = {
  sumar: function(a, b) {
    return a + b;
  },
  restar: function(a, b) {
    return a - b;
  },
  multiplicar: function(a, b) {
    return a * b;
  },
};
```



```

    dividir: function(a, b) {
        if (b === 0) {
            return "Error: división por cero";
        }
        return a / b;
    }
};

console.log("5 + 3 =", calculadora.sumar(5, 3));
console.log("10 - 4 =", calculadora.restar(10, 4));
console.log("6 * 7 =", calculadora.multiplicar(6, 7));
console.log("20 / 5 =", calculadora.dividir(20, 5));

// Objeto con método que usa this
let coche = {
    marca: "Toyota",
    modelo: "Corolla",
    año: 2023,
    descripcion: function() {
        return `${this.marca} ${this.modelo} del año ${this.año}`;
    }
};

console.log(coche.descripcion());

// Object.keys(), Object.values(), Object.entries()
console.log("Claves:", Object.keys(persona));
console.log("Valores:", Object.values(persona));
console.log("Entradas:", Object.entries(persona));

```

Conceptos practicados:

- Creación de objetos literales
- Acceso a propiedades (notación punto y corchetes)
- Modificación y adición de propiedades
- Eliminación de propiedades con `delete`
- Métodos en objetos
- Uso de `this`
- `Object.keys()`, `Object.values()`, `Object.entries()`

Ejercicio 5: Arrays de objetos

Enunciado

Crea un array llamado `estudiantes` con los siguientes objetos:

```
[
  { id: 1, nombre: "Ana", edad: 20, nota: 8.5 },
  { id: 2, nombre: "Carlos", edad: 22, nota: 7.0 },
  { id: 3, nombre: "Elena", edad: 21, nota: 9.2 },
  { id: 4, nombre: "David", edad: 23, nota: 6.8 },
  { id: 5, nombre: "Lucía", edad: 20, nota: 8.9 }
]
```

Realiza las siguientes operaciones:

1. Mostrar el array completo
2. Acceder y mostrar el primer estudiante
3. Acceder y mostrar el nombre del segundo estudiante
4. Añadir un nuevo estudiante: `{ id: 6, nombre: "Miguel", edad: 22, nota: 7.5 }`
5. Crear una función `buscarEstudiante(nombre)` que busque y retorne el estudiante con ese nombre
6. Filtrar y mostrar los estudiantes con nota mayor o igual a 8
7. Calcular y mostrar la nota media de todos los estudiantes
8. Modificar la nota de "Carlos" a 7.5

► Solución

```
// Array de objetos
let estudiantes = [
  { id: 1, nombre: "Ana", edad: 20, nota: 8.5 },
  { id: 2, nombre: "Carlos", edad: 22, nota: 7.0 },
  { id: 3, nombre: "Elena", edad: 21, nota: 9.2 },
  { id: 4, nombre: "David", edad: 23, nota: 6.8 },
  { id: 5, nombre: "Lucía", edad: 20, nota: 8.9 }
];

// Mostrar todos los estudiantes
console.log("Estudiantes:", estudiantes);

// Acceder a un estudiante específico
console.log("Primer estudiante:", estudiantes[0]);
```

```
console.log("Nombre del segundo estudiante:", estudiantes[1].nombre);

// Añadir un nuevo estudiante
estudiantes.push({ id: 6, nombre: "Miguel", edad: 22, nota: 7.5 });
console.log("Después de añadir:", estudiantes);

// Buscar un estudiante por nombre
function buscarEstudiante(nombre) {
    for (let estudiante of estudiantes) {
        if (estudiante.nombre === nombre) {
            return estudiante;
        }
    }
    return null;
}

console.log("Buscar Elena:", buscarEstudiante("Elena"));

// Filtrar estudiantes con nota >= 8
let estudiantesDestacados = [];
for (let estudiante of estudiantes) {
    if (estudiante.nota >= 8) {
        estudiantesDestacados.push(estudiante);
    }
}
console.log("Estudiantes destacados:", estudiantesDestacados);

// Calcular la nota media
let sumaNotas = 0;
for (let estudiante of estudiantes) {
    sumaNotas += estudiante.nota;
}
let notaMedia = sumaNotas / estudiantes.length;
console.log("Nota media:", notaMedia.toFixed(2));

// Modificar la nota de un estudiante
for (let estudiante of estudiantes) {
    if (estudiante.nombre === "Carlos") {
        estudiante.nota = 7.5;
        break;
    }
}
console.log("Después de modificar:", estudiantes);
```

Conceptos practicados:

- Arrays de objetos
 - Acceso a propiedades de objetos dentro de arrays
 - Iteración con `for...of`
 - Búsqueda en arrays de objetos
 - Filtrado de datos
 - Cálculos con datos de arrays
-

Ejercicio 6: Estructuras de control

Realiza los siguientes ejercicios utilizando diferentes estructuras de control:

Parte 1: Condicionales

1. Crea una variable `edad` con valor `18` y usa un `if-else` para mostrar si es mayor o menor de edad
2. Crea una variable `nota` con valor `7.5` y usa `if-else if-else` para mostrar la calificación:
 - Si `nota >= 9`: "Sobresaliente"
 - Si `nota >= 7`: "Notable"
 - Si `nota >= 5`: "Aprobado"
 - Si no: "Suspenso"
3. Crea una variable `día` con valor `3` y usa `switch` para mostrar el nombre del día de la semana
4. Usa el operador ternario para crear una variable que indique si es mayor de edad

Parte 2: Bucles

5. Usa un bucle `for` para mostrar los números del 1 al 5
6. Crea un array `colores` con `["rojo", "verde", "azul", "amarillo"]` y recorre con `for` mostrando el índice y el valor
7. Recorre el array de colores usando `for...of`
8. Usa un bucle `while` para hacer una cuenta atrás desde 5 hasta 1 y mostrar "¡Despegue!"
9. Usa un bucle `do-while` para mostrar los números pares del 0 al 10
10. Usa `break` y `continue` en un bucle que muestre los números del 1 al 10, saltando el 5 y deteniendo en el 9

► Solución



```
// IF-ELSE
let edad = 18;

if (edad >= 18) {
    console.log("Eres mayor de edad");
} else {
    console.log("Eres menor de edad");
}

// IF-ELSE IF-ELSE
let nota = 7.5;

if (nota >= 9) {
    console.log("Sobresaliente");
} else if (nota >= 7) {
    console.log("Notable");
} else if (nota >= 5) {
    console.log("Aprobado");
} else {
    console.log("Suspenso");
}

// SWITCH
let dia = 3;
let nombreDia;

switch (dia) {
    case 1:
        nombreDia = "Lunes";
        break;
    case 2:
        nombreDia = "Martes";
        break;
    case 3:
        nombreDia = "Miércoles";
        break;
    case 4:
        nombreDia = "Jueves";
        break;
    case 5:
        nombreDia = "Viernes";
        break;
    case 6:
        nombreDia = "Sábado";
        break;
    case 7:
```

```
        nombreDia = "Domingo";
        break;
    default:
        nombreDia = "Día no válido";
}

console.log("Día:", nombreDia);

// OPERADOR TERNARIO
let esMayorDeEdad = edad >= 18 ? "Mayor de edad" : "Menor de edad";
console.log(esMayorDeEdad);

// BUCLE FOR
console.log("Números del 1 al 5:");
for (let i = 1; i <= 5; i++) {
    console.log(i);
}

// BUCLE FOR con array
let colores = ["rojo", "verde", "azul", "amarillo"];
console.log("Colores:");
for (let i = 0; i < colores.length; i++) {
    console.log(i + ": " + colores[i]);
}

// BUCLE FOR...OF
console.log("Colores con for...of:");
for (let color of colores) {
    console.log(color);
}

// BUCLE WHILE
console.log("Cuenta atrás:");
let contador = 5;
while (contador > 0) {
    console.log(contador);
    contador--;
}
console.log("¡Despegue!");

// BUCLE DO-WHILE
console.log("Números pares del 0 al 10:");
let numero = 0;
do {
    if (numero % 2 === 0) {
        console.log(numero);
    }
}
```

```
    numero++;  
  } while (numero <= 10);  
  
  // BREAK y CONTINUE  
  console.log("Números del 1 al 10, saltando el 5:");  
  for (let i = 1; i <= 10; i++) {  
    if (i === 5) {  
      continue; // Salta esta iteración  
    }  
    if (i === 9) {  
      break; // Sale del bucle  
    }  
    console.log(i);  
  }  
}
```

Conceptos practicados:

- Condicionales: if, else if, else
- Switch-case
- Operador ternario
- Bucles: for, for...of, while, do-while
- Break y continue

Ejercicio 7: Funciones

Crea las siguientes funciones y puébalas con diferentes valores:

1. **Función declarada** `saludar(nombre)` que retorne un saludo con el nombre
2. **Función declarada** `sumar(a, b)` que retorne la suma de dos números
3. **Función sin return** `mostrarMensaje(mensaje)` que muestre un mensaje por consola
4. **Función con parámetro por defecto** `saludarConIdioma(nombre, idioma = "español")` que salude en diferentes idiomas
5. **Expresión de función** asignada a la variable `multiplicar` que multiplique dos números
6. **Arrow function** asignada a la variable `dividir` que divida dos números (controlar división por cero)
7. **Arrow function simplificada** `cuadrado` que calcule el cuadrado de un número
8. **Arrow function simplificada** `esPar` que determine si un número es par

9. **Función** `crearPersona(nombre, edad)` que retorne un objeto con esas propiedades y un método `saludar()`
10. **Función con rest parameters** `sumarTodos(...numeros)` que sume cualquier cantidad de números
11. **Función recursiva** `factorial(n)` que calcule el factorial de un número

► Solución

```
// Función con declaración
function saludar(nombre) {
    return "Hola, " + nombre + "!";
}

console.log(saludar("Ana"));

// Función con múltiples parámetros
function sumar(a, b) {
    return a + b;
}

console.log("Suma:", sumar(5, 3));

// Función sin return (undefined)
function mostrarMensaje(mensaje) {
    console.log("Mensaje:", mensaje);
}

mostrarMensaje("Esto es una prueba");

// Función con parámetros por defecto
function saludarConIdioma(nombre, idioma = "español") {
    if (idioma === "español") {
        return `Hola, ${nombre}!`;
    } else if (idioma === "inglés") {
        return `Hello, ${nombre}!`;
    } else {
        return `Hi, ${nombre}!`;
    }
}

console.log(saludarConIdioma("Carlos"));
console.log(saludarConIdioma("Carlos", "inglés"));
```

```
// Expresión de función
let multiplicar = function(a, b) {
    return a * b;
};

console.log("Multiplicación:", multiplicar(4, 6));

// Función flecha (arrow function)
let dividir = (a, b) => {
    if (b === 0) {
        return "Error: división por cero";
    }
    return a / b;
};

console.log("División:", dividir(20, 4));

// Arrow function simplificada
let cuadrado = x => x * x;
console.log("Cuadrado de 5:", cuadrado(5));

let esPar = num => num % 2 === 0;
console.log("¿Es 8 par?", esPar(8));
console.log("¿Es 7 par?", esPar(7));

// Función que retorna un objeto
function crearPersona(nombre, edad) {
    return {
        nombre: nombre,
        edad: edad,
        saludar: function() {
            return `Hola, soy ${this.nombre}`;
        }
    };
}

let persona = crearPersona("Luis", 25);
console.log(persona);
console.log(persona.saludar());

// Función con rest parameters
function sumarTodos(...numeros) {
    let suma = 0;
    for (let num of numeros) {
        suma += num;
    }
    return suma;
}
```



```
}

console.log("Suma de varios números:", sumarTodos(1, 2, 3, 4, 5));

// Función recursiva (factorial)
function factorial(n) {
  if (n === 0 || n === 1) {
    return 1;
  }
  return n * factorial(n - 1);
}

console.log("Factorial de 5:", factorial(5));
```

Conceptos practicados:

- Declaración de funciones
- Parámetros y valores de retorno
- Parámetros por defecto
- Expresiones de función
- Arrow functions
- Rest parameters (...)
- Funciones recursivas

Ejercicio 8: Métodos de arrays avanzados

Enunciado

Dado el array `numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, realiza las siguientes operaciones:

1. Usa `map()` para crear un array con los cuadrados de cada número
2. Usa `map()` para crear un array con el doble de cada número
3. Usa `filter()` para obtener solo los números pares
4. Usa `filter()` para obtener solo los números mayores que 5
5. Usa `find()` para encontrar el primer número par
6. Usa `findIndex()` para encontrar el índice del primer número par
7. Usa `some()` para verificar si hay algún número mayor que 5
8. Usa `every()` para verificar si todos los números son mayores que 0

9. Usa `reduce()` para calcular la suma de todos los números
10. Usa `reduce()` para calcular el producto de todos los números

Además, dado el siguiente array de productos:

```
let productos = [  
  { nombre: "Laptop", precio: 800, categoria: "Electrónica" },  
  { nombre: "Mouse", precio: 20, categoria: "Electrónica" },  
  { nombre: "Teclado", precio: 50, categoria: "Electrónica" },  
  { nombre: "Silla", precio: 150, categoria: "Muebles" },  
  { nombre: "Mesa", precio: 300, categoria: "Muebles" }  
];
```

Realiza:

11. Obtener un array solo con los nombres de los productos
12. Filtrar productos de la categoría "Electrónica"
13. Filtrar productos con precio menor a 100
14. Calcular el precio total de todos los productos
15. Encadenar métodos para calcular el total de la categoría "Electrónica"
16. Usar `forEach()` para mostrar todos los productos con formato

► Solución

```
let numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
  
// MAP - Transforma cada elemento  
let cuadrados = numeros.map(num => num * num);  
console.log("Cuadrados:", cuadrados);  
  
let dobles = numeros.map(num => num * 2);  
console.log("Dobles:", dobles);  
  
// FILTER - Filtra elementos  
let pares = numeros.filter(num => num % 2 === 0);  
console.log("Números pares:", pares);  
  
let mayoresQue5 = numeros.filter(num => num > 5);  
console.log("Mayores que 5:", mayoresQue5);  
  
// FIND - Encuentra el primer elemento
```

```
let primerPar = numeros.find(num => num % 2 === 0);
console.log("Primer par:", primerPar);

// FINDINDEX - Encuentra el índice del primer elemento
let indicePrimerPar = numeros.findIndex(num => num % 2 === 0);
console.log("Índice del primer par:", indicePrimerPar);

// SOME - Verifica si al menos uno cumple
let hayMayorQue5 = numeros.some(num => num > 5);
console.log("¿Hay algún número mayor que 5?:", hayMayorQue5);

// EVERY - Verifica si todos cumplen
let todosMayoresQue0 = numeros.every(num => num > 0);
console.log("¿Todos son mayores que 0?:", todosMayoresQue0);

// REDUCE - Reduce a un solo valor
let suma = numeros.reduce((acumulador, num) => acumulador + num, 0);
console.log("Suma de todos:", suma);

let producto = numeros.reduce((acum, num) => acum * num, 1);
console.log("Producto de todos:", producto);

// Ejemplo con array de objetos
let productos = [
  { nombre: "Laptop", precio: 800, categoria: "Electrónica" },
  { nombre: "Mouse", precio: 20, categoria: "Electrónica" },
  { nombre: "Teclado", precio: 50, categoria: "Electrónica" },
  { nombre: "Silla", precio: 150, categoria: "Muebles" },
  { nombre: "Mesa", precio: 300, categoria: "Muebles" }
];

// Obtener solo los nombres
let nombres = productos.map(p => p.nombre);
console.log("Nombres:", nombres);

// Filtrar por categoría
let electronica = productos.filter(p => p.categoria === "Electrónica");
console.log("Electrónica:", electronica);

// Productos baratos (precio < 100)
let baratos = productos.filter(p => p.precio < 100);
console.log("Productos baratos:", baratos);

// Total de precios
let total = productos.reduce((sum, p) => sum + p.precio, 0);
console.log("Precio total:", total);
```

```
// Encadenar métodos
let preciosElectronica = productos
  .filter(p => p.categoria === "Electrónica")
  .map(p => p.precio)
  .reduce((sum, precio) => sum + precio, 0);

console.log("Total electrónica:", preciosElectronica);

// FOREACH - Iterar sin retornar nada
console.log("Lista de productos:");
productos.forEach((producto, index) => {
  console.log(`${index + 1}. ${producto.nombre} - ${producto.precio}`);
});
```

Conceptos practicados:

- map(): transformar arrays
- filter(): filtrar elementos
- find() y findIndex(): búsqueda
- some() y every(): validaciones
- reduce(): reducción a un valor
- forEach(): iteración
- Encadenamiento de métodos

Ejercicios intermedios

Ejercicio 1: Contador interactivo

Crea una página web con un contador que muestre un número y tres botones:

- Un botón "Incrementar" que aumente el contador en 1
- Un botón "Decrementar" que disminuya el contador en 1
- Un botón "Resetear" que vuelva el contador a 0

Estructura HTML requerida:

- Un `<div>` con id `contador` que muestre el número actual
- Tres botones con los ids: `btnIncrementar`, `btnDecrementar`, `btnResetear`

Requisitos adicionales:

- Cuando el contador sea positivo, el texto debe ser verde
- Cuando sea negativo, debe ser rojo
- Cuando sea cero, debe ser negro
- Usa `getElementById()` para acceder a los elementos
- Usa `addEventListener()` para manejar los clics

► Solución propuesta

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contador Interactivo</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    }

    .container {
      background: white;
      padding: 40px;
      border-radius: 15px;
      box-shadow: 0 10px 30px rgba(0,0,0,0.3);
      text-align: center;
    }

    h1 {
      margin-top: 0;
      color: #333;
    }

    #contador {
      font-size: 72px;
      font-weight: bold;
```

```
        margin: 30px 0;
        transition: color 0.3s ease;
    }

    .positivo { color: #28a745; }
    .negativo { color: #dc3545; }
    .cero { color: #333; }

    .botones {
        display: flex;
        gap: 10px;
        justify-content: center;
    }

    button {
        padding: 12px 24px;
        font-size: 16px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        transition: all 0.3s ease;
        font-weight: bold;
    }

    #btnIncrementar {
        background-color: #28a745;
        color: white;
    }

    #btnIncrementar:hover {
        background-color: #218838;
        transform: translateY(-2px);
    }

    #btnDecrementar {
        background-color: #dc3545;
        color: white;
    }

    #btnDecrementar:hover {
        background-color: #c82333;
        transform: translateY(-2px);
    }

    #btnResetear {
        background-color: #6c757d;
        color: white;
    }
```

```

    }

    #btnResetear:hover {
        background-color: #5a6268;
        transform: translateY(-2px);
    }
</style>
</head>
<body>
    <div class="container">
        <h1>Contador Interactivo</h1>
        <div id="contador" class="cero">0</div>
        <div class="botones">
            <button id="btnIncrementar">+ Incrementar</button>
            <button id="btnDecrementar">- Decrementar</button>
            <button id="btnResetear">🔄 Resetear</button>
        </div>
    </div>

    <script>
        // Variable para almacenar el valor del contador
        let valorContador = 0;

        // Obtener referencias a los elementos del DOM
        const contadorElemento = document.getElementById('contador');
        const btnIncrementar = document.getElementById('btnIncrementar');
        const btnDecrementar = document.getElementById('btnDecrementar');
        const btnResetear = document.getElementById('btnResetear');

        // Función para actualizar el display del contador
        function actualizarContador() {
            contadorElemento.textContent = valorContador;

            // Cambiar el color según el valor
            contadorElemento.classList.remove('positivo', 'negativo', 'cero')

            if (valorContador > 0) {
                contadorElemento.classList.add('positivo');
            } else if (valorContador < 0) {
                contadorElemento.classList.add('negativo');
            } else {
                contadorElemento.classList.add('cero');
            }
        }

        // Event listeners para los botones
        btnIncrementar.addEventListener('click', function() {

```

```
        valorContador++;  
        actualizarContador();  
    });  
  
    btnDecrementar.addEventListener('click', function() {  
        valorContador--;  
        actualizarContador();  
    });  
  
    btnResetear.addEventListener('click', function() {  
        valorContador = 0;  
        actualizarContador();  
    });  
</script>  
</body>  
</html>
```

Conceptos practicados:

- `getElementById()` para acceder a elementos
- `addEventListener()` para eventos de click
- Manipulación de `textContent`
- Manipulación de clases con `classList`
- Variables y operadores de incremento/decremento
- Condicionales para cambiar estilos

Ejercicio 2: Lista de tareas simple

Crea una aplicación de lista de tareas donde el usuario pueda:

1. Escribir una tarea en un input
2. Agregar la tarea a una lista al hacer clic en un botón o presionar Enter
3. Cada tarea debe tener un botón "Eliminar" para quitarla de la lista
4. Mostrar un mensaje cuando la lista esté vacía

Estructura HTML requerida:

- Un `<input>` con id `inputTarea` para escribir la tarea
- Un botón con id `btnAgregar` para agregar la tarea

- Un `<ul>` con id `listaTareas` para mostrar las tareas

Requisitos adicionales:

- No permitir agregar tareas vacías
- Limpiar el input después de agregar una tarea
- Cada tarea debe crearse con `createElement()` y `appendChild()`

► Solución

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lista de Tareas</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: 50px auto;
      padding: 20px;
      background-color: #f5f5f5;
    }

    .container {
      background: white;
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    }

    h1 {
      color: #333;
      text-align: center;
      margin-top: 0;
    }

    .input-container {
      display: flex;
      gap: 10px;
      margin-bottom: 20px;
    }
```

```
#inputTarea {
  flex: 1;
  padding: 12px;
  border: 2px solid #ddd;
  border-radius: 5px;
  font-size: 16px;
}

#inputTarea:focus {
  outline: none;
  border-color: #007bff;
}

#btnAgregar {
  padding: 12px 24px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
  font-weight: bold;
}

#btnAgregar:hover {
  background-color: #0056b3;
}

#listaTareas {
  list-style: none;
  padding: 0;
}

.tarea-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px;
  margin-bottom: 10px;
  background-color: #f8f9fa;
  border-left: 4px solid #007bff;
  border-radius: 5px;
  animation: aparecer 0.3s ease;
}

@keyframes aparecer {
```

```

        from {
            opacity: 0;
            transform: translateX(-20px);
        }
        to {
            opacity: 1;
            transform: translateX(0);
        }
    }

    .tarea-texto {
        flex: 1;
        color: #333;
    }

    .btn-eliminar {
        padding: 8px 16px;
        background-color: #dc3545;
        color: white;
        border: none;
        border-radius: 3px;
        cursor: pointer;
        font-size: 14px;
    }

    .btn-eliminar:hover {
        background-color: #c82333;
    }

    .mensaje-vacio {
        text-align: center;
        color: #999;
        padding: 40px;
        font-style: italic;
    }
</style>
</head>
<body>
    <div class="container">
        <h1> 📌 Mi Lista de Tareas</h1>

        <div class="input-container">
            <input type="text" id="inputTarea" placeholder="Escribe una nuev
            <button id="btnAgregar">Agregar</button>
        </div>

        <ul id="listaTareas">

```

```
        <div class="mensaje-vacio">No hay tareas. ¡Agrega tu primera tar
    </ul>
</div>

<script>
    // Obtener referencias a los elementos
    const inputTarea = document.getElementById('inputTarea');
    const btnAgregar = document.getElementById('btnAgregar');
    const listaTareas = document.getElementById('listaTareas');

    // Función para agregar una tarea
    function agregarTarea() {
        const textoTarea = inputTarea.value.trim();

        // Validar que no esté vacío
        if (textoTarea === '') {
            alert('Por favor, escribe una tarea');
            return;
        }

        // Eliminar mensaje vacío si existe
        const mensajeVacio = listaTareas.querySelector('.mensaje-vacio');
        if (mensajeVacio) {
            mensajeVacio.remove();
        }

        // Crear elemento li
        const li = document.createElement('li');
        li.className = 'tarea-item';

        // Crear span para el texto
        const span = document.createElement('span');
        span.className = 'tarea-texto';
        span.textContent = textoTarea;

        // Crear botón eliminar
        const btnEliminar = document.createElement('button');
        btnEliminar.className = 'btn-eliminar';
        btnEliminar.textContent = 'Eliminar';

        // Agregar evento al botón eliminar
        btnEliminar.addEventListener('click', function() {
            li.remove();

            // Si no quedan tareas, mostrar mensaje
            if (listaTareas.children.length === 0) {
                mostrarMensajeVacio();
            }
        });
    }
}
```

```

        }
    });

    // Agregar elementos al li
    li.appendChild(span);
    li.appendChild(btnEliminar);

    // Agregar li a la lista
    listaTareas.appendChild(li);

    // Limpiar el input
    inputTarea.value = '';
    inputTarea.focus();
}

// Función para mostrar mensaje cuando no hay tareas
function mostrarMensajeVacio() {
    const div = document.createElement('div');
    div.className = 'mensaje-vacio';
    div.textContent = 'No hay tareas. ¡Agrega tu primera tarea!';
    listaTareas.appendChild(div);
}

// Event listener para el botón
btnAgregar.addEventListener('click', agregarTarea);

// Event listener para la tecla Enter
inputTarea.addEventListener('keypress', function(evento) {
    if (evento.key === 'Enter') {
        agregarTarea();
    }
});
</script>
</body>
</html>

```

Conceptos practicados:

- `createElement()` para crear elementos
- `appendChild()` para agregar elementos al DOM
- `remove()` para eliminar elementos
- `querySelector()` para buscar elementos
- Eventos de teclado (`keypress`)

- Validación de inputs
- Manipulación de clases y estilos

Ejercicio 3: Cambiar colores dinámicamente

Crea una página que permita cambiar el color de fondo de un elemento mediante botones y un input de texto:

1. Un `<div>` grande que cambiará de color
2. Tres botones con colores predefinidos (Rojo, Verde, Azul)
3. Un input donde el usuario pueda escribir cualquier color válido (nombre o código hex)
4. Un botón "Aplicar color personalizado"
5. Un botón "Color aleatorio" que genere un color random

Estructura HTML requerida:

- Un `<div>` con id `cajaColor` que será el elemento a colorear
- Botones para colores predefinidos
- Un `<input>` con id `inputColor` para colores personalizados
- Mostrar el código del color actual en un `<span>` con id `colorActual`

► Solución

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cambiar Colores</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
      background-color: #f0f0f0;
    }

    .container {
      max-width: 600px;
      margin: 0 auto;
    }
  </style>
</head>
<body>
  <div id="cajaColor">
    <div id="botones">
      <button>Rojo</button>
      <button>Verde</button>
      <button>Azul</button>
    </div>
    <input id="inputColor" type="text" value="Color personalizado">
    <button id="aplicar" type="button">Aplicar color personalizado</button>
    <button id="aleatorio" type="button">Color aleatorio</button>
    <span id="colorActual"></span>
  </div>
</body>
</html>
```

```
        background: white;
        padding: 30px;
        border-radius: 10px;
        box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    }

    h1 {
        text-align: center;
        color: #333;
    }

    #cajaColor {
        width: 100%;
        height: 300px;
        border: 3px solid #333;
        border-radius: 10px;
        margin: 20px 0;
        transition: background-color 0.3s ease;
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 24px;
        color: white;
        text-shadow: 2px 2px 4px rgba(0,0,0,0.5);
        background-color: #6c757d;
    }

    .info-color {
        text-align: center;
        margin-bottom: 20px;
        font-size: 18px;
    }

    #colorActual {
        font-weight: bold;
        color: #007bff;
    }

    .botones-predefinidos {
        display: flex;
        gap: 10px;
        margin-bottom: 20px;
        flex-wrap: wrap;
    }

    .btn-color {
        flex: 1;
```

```
        min-width: 100px;
        padding: 15px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        font-size: 16px;
        font-weight: bold;
        color: white;
        transition: transform 0.2s ease;
    }

    .btn-color:hover {
        transform: scale(1.05);
    }

    .btn-rojo {
        background-color: #dc3545;
    }

    .btn-verde {
        background-color: #28a745;
    }

    .btn-azul {
        background-color: #007bff;
    }

    .btn-amarillo {
        background-color: #ffc107;
        color: #333;
    }

    .input-personalizado {
        display: flex;
        gap: 10px;
        margin-bottom: 20px;
    }

    #inputColor {
        flex: 1;
        padding: 12px;
        border: 2px solid #ddd;
        border-radius: 5px;
        font-size: 16px;
    }

    #btnAplicar, #btnAleatorio {
```



```

        padding: 12px 24px;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        font-size: 16px;
        font-weight: bold;
        color: white;
    }

    #btnAplicar {
        background-color: #6c757d;
    }

    #btnAplicar:hover {
        background-color: #5a6268;
    }

    #btnAleatorio {
        background-color: #17a2b8;
        width: 100%;
    }

    #btnAleatorio:hover {
        background-color: #138496;
    }
</style>
</head>
<body>
    <div class="container">
        <h1>🎨 Cambiador de Colores</h1>

        <div id="cajaColor">
            Color de Fondo
        </div>

        <div class="info-color">
            Color actual: <span id="colorActual">#6c757d</span>
        </div>

        <h3>Colores Predefinidos:</h3>
        <div class="botones-predefinidos">
            <button class="btn-color btn-rojo" onclick="cambiarColor('#dc3545')>Rojo
            <button class="btn-color btn-verde" onclick="cambiarColor('#28a745')>Verde
            <button class="btn-color btn-azul" onclick="cambiarColor('#007bff')>Azul
            <button class="btn-color btn-amarillo" onclick="cambiarColor('#ffc107')>Amarillo
        </div>
    </div>

```

```

<h3>Color Personalizado:</h3>
<div class="input-personalizado">
  <input type="text" id="inputColor" placeholder="Escribe un color"
  <button id="btnAplicar">Aplicar</button>
</div>

  <button id="btnAleatorio">🎲 Color Aleatorio</button>
</div>

<script>
  // Obtener referencias a los elementos
  const cajaColor = document.getElementById('cajaColor');
  const inputColor = document.getElementById('inputColor');
  const btnAplicar = document.getElementById('btnAplicar');
  const btnAleatorio = document.getElementById('btnAleatorio');
  const colorActual = document.getElementById('colorActual');

  // Función para cambiar el color
  function cambiarColor(color) {
    cajaColor.style.backgroundColor = color;
    colorActual.textContent = color;
  }

  // Función para aplicar color personalizado
  function aplicarColorPersonalizado() {
    const color = inputColor.value.trim();

    if (color === '') {
      alert('Por favor, escribe un color');
      return;
    }

    // Intentar aplicar el color
    cambiarColor(color);
    inputColor.value = '';
  }

  // Función para generar color aleatorio
  function generarColorAleatorio() {
    // Generar valores RGB aleatorios
    const r = Math.floor(Math.random() * 256);
    const g = Math.floor(Math.random() * 256);
    const b = Math.floor(Math.random() * 256);

    // Convertir a formato hexadecimal
    const colorHex = '#' +
      r.toString(16).padStart(2, '0') +

```

```

        g.toString(16).padStart(2, '0') +
        b.toString(16).padStart(2, '0');

        cambiarColor(colorHex);
    }

    // Event listeners
    btnAplicar.addEventListener('click', aplicarColorPersonalizado);

    btnAleatorio.addEventListener('click', generarColorAleatorio);

    // Permitir aplicar con Enter
    inputColor.addEventListener('keypress', function(evento) {
        if (evento.key === 'Enter') {
            aplicarColorPersonalizado();
        }
    });
</script>
</body>
</html>

```

Conceptos practicados:

- Manipulación de `style` para cambiar estilos inline
- `Math.random()` y `Math.floor()` para números aleatorios
- Conversión de números a hexadecimal con `toString(16)`
- `padStart()` para formatear strings
- Funciones globales y event listeners

Ejercicio 4: Mostrar/Ocultar contenido

Crea una página con tres secciones de contenido que se puedan mostrar u ocultar:

1. Tres botones, cada uno corresponde a una sección
2. Al hacer clic en un botón, se muestra su sección y se ocultan las demás
3. Agregar un botón "Mostrar todo" que muestre las tres secciones
4. Agregar un botón "Ocultar todo" que oculte las tres secciones

Estructura HTML requerida:

- Tres `<div>` con ids `seccion1`, `seccion2`, `seccion3`

- Botones para controlar cada sección
- Botones para mostrar/ocultar todo

Requisitos adicionales:

- Usar `classList.add()` y `classList.remove()` para manipular clases
- Crear una clase CSS `.oculto` con `display: none`
- Los botones activos deben tener un estilo diferente

► Solución

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mostrar/Ocultar Contenido</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 800px;
      margin: 50px auto;
      padding: 20px;
      background-color: #f5f5f5;
    }

    .container {
      background: white;
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    }

    h1 {
      text-align: center;
      color: #333;
    }

    .controles {
      margin-bottom: 30px;
    }

    .grupo-botones {
```

```
        display: flex;
        gap: 10px;
        margin-bottom: 15px;
        flex-wrap: wrap;
    }

    button {
        padding: 12px 24px;
        border: 2px solid #007bff;
        background-color: white;
        color: #007bff;
        border-radius: 5px;
        cursor: pointer;
        font-size: 16px;
        font-weight: bold;
        transition: all 0.3s ease;
    }

    button:hover {
        background-color: #e7f3ff;
    }

    button.activo {
        background-color: #007bff;
        color: white;
    }

    .btn-secundario {
        border-color: #6c757d;
        color: #6c757d;
    }

    .btn-secundario:hover {
        background-color: #f0f0f0;
    }

    .seccion {
        padding: 25px;
        margin-bottom: 20px;
        border-radius: 8px;
        border-left: 5px solid;
        transition: all 0.3s ease;
    }

    #seccion1 {
        background-color: #fff3cd;
        border-left-color: #ffc107;
```

```

    }

    #seccion2 {
        background-color: #d1ecf1;
        border-left-color: #17a2b8;
    }

    #seccion3 {
        background-color: #d4edda;
        border-left-color: #28a745;
    }

    .seccion h2 {
        margin-top: 0;
        color: #333;
    }

    .seccion p {
        line-height: 1.6;
        color: #666;
    }

    .oculto {
        display: none;
    }

    .divisor {
        margin: 20px 0;
        border: 0;
        height: 1px;
        background-color: #ddd;
    }
</style>
</head>
<body>
    <div class="container">
        <h1> 📄 Panel de Secciones</h1>

        <div class="controles">
            <div class="grupo-botones">
                <button id="btnSeccion1" onclick="mostrarSeccion(1)">Mostrar
                <button id="btnSeccion2" onclick="mostrarSeccion(2)">Mostrar
                <button id="btnSeccion3" onclick="mostrarSeccion(3)">Mostrar
            </div>

            <hr class="divisor">

```

```

        <div class="grupo-botones">
            <button class="btn-secundario" onclick="mostrarTodas()"><img alt="ojo" data-bbox="860 75 880 90"/>
            <button class="btn-secundario" onclick="ocultarTodas()"><img alt="candado" data-bbox="860 95 880 110"/>
        </div>
    </div>

    <div id="seccion1" class="seccion">
        <h2><img alt="libro" data-bbox="290 190 310 205"/> Sección 1: Introducción</h2>
        <p>
            Esta es la primera sección de contenido. Contiene información
            sobre el tema que estamos tratando. El contenido puede ser manipulado
            dinámicamente usando JavaScript.
        </p>
        <p>
            La manipulación del DOM nos permite controlar la visibilidad de los elementos
            de forma interactiva.
        </p>
    </div>

    <div id="seccion2" class="seccion">
        <h2><img alt="bombilla" data-bbox="290 435 310 450"/> Sección 2: Desarrollo</h2>
        <p>
            En esta segunda sección desarrollamos el contenido principal de la aplicación
            en los conceptos y explicamos los detalles más importantes.
        </p>
        <p>
            Las clases CSS y JavaScript trabajan juntos para crear una experiencia
            dinámica y atractiva.
        </p>
    </div>

    <div id="seccion3" class="seccion">
        <h2><img alt="check" data-bbox="290 665 310 680"/> Sección 3: Conclusión</h2>
        <p>
            Esta tercera sección contiene las conclusiones y el resumen de la aplicación.
            Es importante destacar los puntos clave y las ideas principales.
        </p>
        <p>
            Con estas técnicas podemos crear interfaces más interactivas y mejorar la
            experiencia del usuario en nuestras aplicaciones web.
        </p>
    </div>
</div>

<script>
    // Obtener referencias a los elementos
    const seccion1 = document.getElementById('seccion1');
```

```
const seccion2 = document.getElementById('seccion2');
const seccion3 = document.getElementById('seccion3');

const btnSeccion1 = document.getElementById('btnSeccion1');
const btnSeccion2 = document.getElementById('btnSeccion2');
const btnSeccion3 = document.getElementById('btnSeccion3');

// Función para mostrar una sección específica
function mostrarSeccion(numero) {
    // Ocultar todas las secciones
    seccion1.classList.add('oculto');
    seccion2.classList.add('oculto');
    seccion3.classList.add('oculto');

    // Desactivar todos los botones
    btnSeccion1.classList.remove('activo');
    btnSeccion2.classList.remove('activo');
    btnSeccion3.classList.remove('activo');

    // Mostrar la sección seleccionada y activar su botón
    switch(numero) {
        case 1:
            seccion1.classList.remove('oculto');
            btnSeccion1.classList.add('activo');
            break;
        case 2:
            seccion2.classList.remove('oculto');
            btnSeccion2.classList.add('activo');
            break;
        case 3:
            seccion3.classList.remove('oculto');
            btnSeccion3.classList.add('activo');
            break;
    }
}

// Función para mostrar todas las secciones
function mostrarTodas() {
    seccion1.classList.remove('oculto');
    seccion2.classList.remove('oculto');
    seccion3.classList.remove('oculto');

    // Activar todos los botones
    btnSeccion1.classList.add('activo');
    btnSeccion2.classList.add('activo');
    btnSeccion3.classList.add('activo');
}
```



```
// Función para ocultar todas las secciones
function ocultarTodas() {
    seccion1.classList.add('oculto');
    seccion2.classList.add('oculto');
    seccion3.classList.add('oculto');

    // Desactivar todos los botones
    btnSeccion1.classList.remove('activo');
    btnSeccion2.classList.remove('activo');
    btnSeccion3.classList.remove('activo');
}
</script>
</body>
</html>
```

Conceptos practicados:

- `classList.add()` y `classList.remove()` para manipular clases
 - Mostrar/ocultar elementos con CSS
 - Switch-case para control de flujo
 - Múltiples event handlers
 - Gestión de estados visuales (botones activos)
-

Ejercicio 5: Formulario con validación

Enunciado

Crea un formulario de registro con validación en tiempo real:

Campos del formulario:

1. Nombre (mínimo 3 caracteres)
2. Email (formato válido)
3. Edad (entre 18 y 100)
4. Contraseña (mínimo 6 caracteres)

Requisitos:

- Validar cada campo cuando pierda el foco (`blur event`)

- Mostrar mensajes de error debajo de cada campo inválido
- Deshabilitar el botón de envío si hay errores
- Mostrar un resumen de los datos cuando el formulario sea válido

Estructura HTML requerida:

- Un `<form>` con id `formularioRegistro`
- Inputs con ids: `nombre`, `email`, `edad`, `password`
- `<span>` con clase `error` debajo de cada input para mensajes de error
- Un botón de envío con id `btnEnviar`
- Un `<div>` con id `resumen` para mostrar los datos

► Solución

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulario con Validación</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: 50px auto;
      padding: 20px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    }

    .container {
      background: white;
      padding: 40px;
      border-radius: 10px;
      box-shadow: 0 10px 30px rgba(0,0,0,0.3);
    }

    h1 {
      text-align: center;
      color: #333;
      margin-top: 0;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Formulario de Registro</h1>
    <form id="formularioRegistro">
      <div>
        <input type="text" id="nombre" />
        <span id="errorNombre"></span>
      </div>
      <div>
        <input type="email" id="email" />
        <span id="errorEmail"></span>
      </div>
      <div>
        <input type="text" id="edad" />
        <span id="errorEdad"></span>
      </div>
      <div>
        <input type="password" id="password" />
        <span id="errorPassword"></span>
      </div>
      <div>
        <input type="button" value="Enviar" id="btnEnviar" />
      </div>
      <div id="resumen"></div>
    </form>
  </div>
</body>
</html>
```

```
.form-group {
    margin-bottom: 25px;
}

label {
    display: block;
    margin-bottom: 8px;
    font-weight: bold;
    color: #333;
}

input {
    width: 100%;
    padding: 12px;
    border: 2px solid #ddd;
    border-radius: 5px;
    font-size: 16px;
    box-sizing: border-box;
    transition: border-color 0.3s ease;
}

input:focus {
    outline: none;
    border-color: #667eea;
}

input.invalido {
    border-color: #dc3545;
}

input.valido {
    border-color: #28a745;
}

.error {
    display: block;
    color: #dc3545;
    font-size: 14px;
    margin-top: 5px;
    min-height: 20px;
}

#btnEnviar {
    width: 100%;
    padding: 15px;
    background-color: #667eea;
    color: white;
```

```

        border: none;
        border-radius: 5px;
        font-size: 18px;
        font-weight: bold;
        cursor: pointer;
        transition: background-color 0.3s ease;
    }

    #btnEnviar:hover:not(:disabled) {
        background-color: #764ba2;
    }

    #btnEnviar:disabled {
        background-color: #ccc;
        cursor: not-allowed;
    }

    #resumen {
        margin-top: 30px;
        padding: 20px;
        background-color: #d4edda;
        border-left: 5px solid #28a745;
        border-radius: 5px;
        display: none;
    }

    #resumen h2 {
        margin-top: 0;
        color: #28a745;
    }

    #resumen p {
        margin: 10px 0;
        color: #333;
    }

    #resumen strong {
        color: #155724;
    }
</style>
</head>
<body>
    <div class="container">
        <h1>📄 Formulario de Registro</h1>

        <form id="formularioRegistro">
            <div class="form-group">

```

```

        <label for="nombre">Nombre completo:</label>
        <input type="text" id="nombre" placeholder="Ej: Juan Pérez">
        <span class="error" id="errorNombre"></span>
    </div>

    <div class="form-group">
        <label for="email">Correo electrónico:</label>
        <input type="email" id="email" placeholder="Ej: usuario@ejem"
        <span class="error" id="errorEmail"></span>
    </div>

    <div class="form-group">
        <label for="edad">Edad:</label>
        <input type="number" id="edad" placeholder="Ej: 25">
        <span class="error" id="errorEdad"></span>
    </div>

    <div class="form-group">
        <label for="password">Contraseña:</label>
        <input type="password" id="password" placeholder="Mínimo 6 c
        <span class="error" id="errorPassword"></span>
    </div>

    <button type="submit" id="btnEnviar" disabled>Registrarse</button>
</form>

<div id="resumen">
    <h2>✅ Registro Exitoso</h2>
    <p><strong>Nombre:</strong> <span id="resumenNombre"></span></p>
    <p><strong>Email:</strong> <span id="resumenEmail"></span></p>
    <p><strong>Edad:</strong> <span id="resumenEdad"></span></p>
</div>
</div>

<script>
    // Obtener referencias a los elementos
    const formulario = document.getElementById('formularioRegistro');
    const inputNombre = document.getElementById('nombre');
    const inputEmail = document.getElementById('email');
    const inputEdad = document.getElementById('edad');
    const inputPassword = document.getElementById('password');
    const btnEnviar = document.getElementById('btnEnviar');
    const resumen = document.getElementById('resumen');

    // Mensajes de error
    const errorNombre = document.getElementById('errorNombre');
    const errorEmail = document.getElementById('errorEmail');

```

```
const errorEdad = document.getElementById('errorEdad');
const errorPassword = document.getElementById('errorPassword');

// Estado de validación
let validacion = {
  nombre: false,
  email: false,
  edad: false,
  password: false
};

// Función para validar nombre
function validarNombre() {
  const valor = inputNombre.value.trim();

  if (valor.length < 3) {
    errorNombre.textContent = 'El nombre debe tener al menos 3 c
    inputNombre.classList.add('invalido');
    inputNombre.classList.remove('valido');
    validacion.nombre = false;
  } else {
    errorNombre.textContent = '';
    inputNombre.classList.remove('invalido');
    inputNombre.classList.add('valido');
    validacion.nombre = true;
  }

  actualizarBotonEnviar();
}

// Función para validar email
function validarEmail() {
  const valor = inputEmail.value.trim();
  const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  if (!regexEmail.test(valor)) {
    errorEmail.textContent = 'Ingresa un email válido';
    inputEmail.classList.add('invalido');
    inputEmail.classList.remove('valido');
    validacion.email = false;
  } else {
    errorEmail.textContent = '';
    inputEmail.classList.remove('invalido');
    inputEmail.classList.add('valido');
    validacion.email = true;
  }
}
```

```

    actualizarBotonEnviar();
}

// Función para validar edad
function validarEdad() {
    const valor = parseInt(inputEdad.value);

    if (isNaN(valor) || valor < 18 || valor > 100) {
        errorEdad.textContent = 'La edad debe estar entre 18 y 100 años';
        inputEdad.classList.add('invalido');
        inputEdad.classList.remove('valido');
        validacion.edad = false;
    } else {
        errorEdad.textContent = '';
        inputEdad.classList.remove('invalido');
        inputEdad.classList.add('valido');
        validacion.edad = true;
    }

    actualizarBotonEnviar();
}

// Función para validar contraseña
function validarPassword() {
    const valor = inputPassword.value;

    if (valor.length < 6) {
        errorPassword.textContent = 'La contraseña debe tener al menos 6 caracteres';
        inputPassword.classList.add('invalido');
        inputPassword.classList.remove('valido');
        validacion.password = false;
    } else {
        errorPassword.textContent = '';
        inputPassword.classList.remove('invalido');
        inputPassword.classList.add('valido');
        validacion.password = true;
    }

    actualizarBotonEnviar();
}

// Función para actualizar el botón de envío
function actualizarBotonEnviar() {
    const formularioValido = validacion.nombre &&
        validacion.email &&
        validacion.edad &&
        validacion.password;

```

```
        btnEnviar.disabled = !formularioValido;
    }

    // Event listeners para validación en tiempo real
    inputNombre.addEventListener('blur', validarNombre);
    inputEmail.addEventListener('blur', validarEmail);
    inputEdad.addEventListener('blur', validarEdad);
    inputPassword.addEventListener('blur', validarPassword);

    // También validar mientras se escribe
    inputNombre.addEventListener('input', validarNombre);
    inputEmail.addEventListener('input', validarEmail);
    inputEdad.addEventListener('input', validarEdad);
    inputPassword.addEventListener('input', validarPassword);

    // Manejar el envío del formulario
    formulario.addEventListener('submit', function(evento) {
        evento.preventDefault();

        // Mostrar resumen
        document.getElementById('resumenNombre').textContent = inputNombre.value;
        document.getElementById('resumenEmail').textContent = inputEmail.value;
        document.getElementById('resumenEdad').textContent = inputEdad.value;

        resumen.style.display = 'block';

        // Limpiar formulario
        formulario.reset();

        // Resetear validación
        validacion = {
            nombre: false,
            email: false,
            edad: false,
            password: false
        };

        // Limpiar clases
        inputNombre.classList.remove('valido', 'invalido');
        inputEmail.classList.remove('valido', 'invalido');
        inputEdad.classList.remove('valido', 'invalido');
        inputPassword.classList.remove('valido', 'invalido');

        // Deshabilitar botón
        btnEnviar.disabled = true;
    });
```



```
</script>
</body>
</html>
```

Conceptos practicados:

- Eventos `blur` e `input` para validación
- Expresiones regulares (regex) para validar email
- `preventDefault()` para controlar el envío del formulario
- Gestión de estado con objetos
- Manipulación de atributos (`disabled`)
- Validación de formularios en tiempo real

Ejercicios de Proyecto: Calculadora y Lista de Tareas

Ejercicio 1: Calculadora Básica

Crear una calculadora básica que permita realizar operaciones de suma, resta, multiplicación y división entre dos números.

A continuación se muestra el código HTML y CSS base. Deberás completar el código JavaScript para que la calculadora funcione correctamente.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Calculadora Básica</title>
  <style>
    .calculadora {
      max-width: 300px;
      margin: 50px auto;
      padding: 20px;
      border: 2px solid #333;
      border-radius: 10px;
      background-color: #f0f0f0;
    }
  </style>
</head>
</html>
```

```
.input-group {
    margin-bottom: 15px;
}

label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

input[type="number"] {
    width: 100%;
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
}

.botonos {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 10px;
    margin: 20px 0;
}

button {
    padding: 15px;
    font-size: 18px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    background-color: #007bff;
    color: white;
}

button:hover {
    background-color: #0056b3;
}

#resultado {
    font-size: 24px;
    font-weight: bold;
    text-align: center;
    padding: 15px;
    background-color: white;
    border: 2px solid #333;
    border-radius: 5px;
}
```

```

        margin-top: 15px;
    }

    .error {
        color: red;
    }
</style>
</head>
<body>
    <div class="calculadora">
        <h2>Calculadora Básica</h2>

        <div class="input-group">
            <label for="numero1">Primer número:</label>
            <input type="number" id="numero1" step="any">
        </div>

        <div class="input-group">
            <label for="numero2">Segundo número:</label>
            <input type="number" id="numero2" step="any">
        </div>

        <div class="botones">
            <button onclick="calcular('+')">+</button>
            <button onclick="calcular('-')">-</button>
            <button onclick="calcular('*')">*</button>
            <button onclick="calcular('/')">/</button>
        </div>

        <button onclick="limpiar()" style="width: 100%; background-color: #dc3545">

        <div id="resultado">Resultado: 0</div>
    </div>

    <script>
        // Aquí va el código JavaScript para la calculadora
    </script>
</body>
</html>

```

► Solución JavaScript

```

function calcular(operacion) {
    const num1 = parseFloat(document.getElementById('numero1').value);

```



```
const num2 = parseFloat(document.getElementById('numero2').value);
const resultadoDiv = document.getElementById('resultado');

// Validar que ambos números sean válidos
if (isNaN(num1) || isNaN(num2)) {
    resultadoDiv.textContent = 'Error: Ingrese números válidos';
    resultadoDiv.className = 'error';
    return;
}

let resultado;

switch (operacion) {
    case '+':
        resultado = num1 + num2;
        break;
    case '-':
        resultado = num1 - num2;
        break;
    case '*':
        resultado = num1 * num2;
        break;
    case '/':
        if (num2 === 0) {
            resultadoDiv.textContent = 'Error: No se puede dividir por 0';
            resultadoDiv.className = 'error';
            return;
        }
        resultado = num1 / num2;
        break;
    default:
        resultadoDiv.textContent = 'Error: Operación no válida';
        resultadoDiv.className = 'error';
        return;
}

// Mostrar resultado
resultadoDiv.textContent = `Resultado: ${resultado.toFixed(2)}`;
resultadoDiv.className = '';
}

function limpiar() {
    document.getElementById('numero1').value = '';
    document.getElementById('numero2').value = '';
    document.getElementById('resultado').textContent = 'Resultado: 0';
    document.getElementById('resultado').className = '';
}
```

```
// Permitir calcular con Enter
document.addEventListener('keydown', (e) => {
  if (e.key === 'Enter') {
    const ultimaOperacion = '+'; // Por defecto suma
    calcular(ultimaOperacion);
  }
});
```

Ejercicio 2: Lista de Tareas Interactiva

Crear una aplicación de lista de tareas que permita agregar, marcar como completadas y eliminar tareas. Las tareas completadas deben mostrarse con un estilo diferente.

► Solución

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lista de Tareas</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 600px;
      margin: 0 auto;
      padding: 20px;
      background-color: #f5f5f5;
    }

    .container {
      background-color: white;
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }

    h1 {
      text-align: center;
      color: #333;
      margin-bottom: 30px;
```

```
}

.input-section {
  display: flex;
  margin-bottom: 30px;
}

#nuevaTarea {
  flex: 1;
  padding: 12px;
  border: 2px solid #ddd;
  border-radius: 5px 0 0 5px;
  font-size: 16px;
}

#nuevaTarea:focus {
  outline: none;
  border-color: #007bff;
}

#agregarBtn {
  padding: 12px 20px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 0 5px 5px 0;
  cursor: pointer;
  font-size: 16px;
}

#agregarBtn:hover {
  background-color: #0056b3;
}

.estadisticas {
  display: flex;
  justify-content: space-between;
  margin-bottom: 20px;
  padding: 15px;
  background-color: #f8f9fa;
  border-radius: 5px;
}

.stat {
  text-align: center;
}
```

```
.stat-number {
    font-size: 24px;
    font-weight: bold;
    color: #007bff;
}

.stat-label {
    font-size: 14px;
    color: #666;
}

#listaTareas {
    list-style: none;
    padding: 0;
}

.tarea {
    display: flex;
    align-items: center;
    padding: 15px;
    border: 1px solid #ddd;
    border-radius: 5px;
    margin-bottom: 10px;
    background-color: #fff;
    transition: all 0.3s ease;
}

.tarea:hover {
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

.tarea.completada {
    background-color: #f8f9fa;
    opacity: 0.7;
}

.tarea.completada .texto-tarea {
    text-decoration: line-through;
    color: #6c757d;
}

.texto-tarea {
    flex: 1;
    margin-left: 10px;
    font-size: 16px;
}
```

```
.checkbox-tarea {
    width: 20px;
    height: 20px;
    cursor: pointer;
}

.btn-eliminar {
    background-color: #dc3545;
    color: white;
    border: none;
    padding: 8px 12px;
    border-radius: 3px;
    cursor: pointer;
    font-size: 14px;
}

.btn-eliminar:hover {
    background-color: #c82333;
}

.empty-state {
    text-align: center;
    padding: 40px;
    color: #6c757d;
}

.controles {
    margin-top: 20px;
    text-align: center;
}

.controles button {
    margin: 0 5px;
    padding: 8px 15px;
    border: 1px solid #007bff;
    background-color: white;
    color: #007bff;
    border-radius: 3px;
    cursor: pointer;
}

.controles button:hover {
    background-color: #007bff;
    color: white;
}
</style>
</head>
```



```

<body>
  <div class="container">
    <h1> 📌 Mi Lista de Tareas</h1>

    <div class="input-section">
      <input type="text" id="nuevaTarea" placeholder="Escribe una nuev
      <button id="agregarBtn">Agregar</button>
    </div>

    <div class="estadisticas">
      <div class="stat">
        <div class="stat-number" id="totalTareas">0</div>
        <div class="stat-label">Total</div>
      </div>
      <div class="stat">
        <div class="stat-number" id="tareasCompletadas">0</div>
        <div class="stat-label">Completadas</div>
      </div>
      <div class="stat">
        <div class="stat-number" id="tareasPendientes">0</div>
        <div class="stat-label">Pendientes</div>
      </div>
    </div>

    <ul id="listaTareas"></ul>

    <div class="controles">
      <button onclick="mostrarTodas()">Todas</button>
      <button onclick="mostrarPendientes()">Pendientes</button>
      <button onclick="mostrarCompletadas()">Completadas</button>
      <button onclick="eliminarCompletadas()">Eliminar Completadas</bu
    </div>
  </div>

  <script>
    class GestorTareas {
      constructor() {
        this.tareas = JSON.parse(localStorage.getItem('tareas')) ||
        this.filtro = 'todas'; // 'todas', 'pendientes', 'completadas'
        this.inicializar();
      }

      inicializar() {
        this.configurarEventos();
        this.renderizar();
        this.actualizarEstadisticas();
      }
    }
  </script>

```

```

configurarEventos() {
  const input = document.getElementById('nuevaTarea');
  const botonAgregar = document.getElementById('agregarBtn');

  // Agregar tarea con botón o Enter
  botonAgregar.addEventListener('click', () => this.agregarTarea());
  input.addEventListener('keypress', (e) => {
    if (e.key === 'Enter') {
      this.agregarTarea();
    }
  });

  // Delegación de eventos para la lista
  document.getElementById('listaTareas').addEventListener('click', (e) => {
    const li = e.target.closest('.tarea');
    if (!li) return;

    const id = parseInt(li.dataset.id);

    if (e.target.classList.contains('checkbox-tarea')) {
      this.toggleCompletada(id);
    } else if (e.target.classList.contains('btn-eliminar')) {
      this.eliminarTarea(id);
    }
  });
}

agregarTarea() {
  const input = document.getElementById('nuevaTarea');
  const texto = input.value.trim();

  if (texto === '') {
    alert('Por favor, escribe una tarea');
    return;
  }

  const nuevaTarea = {
    id: Date.now(),
    texto: texto,
    completada: false,
    fechaCreacion: new Date().toISOString()
  };

  this.tareas.push(nuevaTarea);
  input.value = '';
  this.guardar();
}

```

```

        this.renderizar();
        this.actualizarEstadisticas();

        // Animación de entrada
        setTimeout(() => {
            const ultimoElemento = document.querySelector(`[data-id=
            if (ultimoElemento) {
                ultimoElemento.style.transform = 'scale(1.05)';
                setTimeout(() => {
                    ultimoElemento.style.transform = 'scale(1)';
                }, 200);
            }
        }, 100);
    }

    toggleCompletada(id) {
        const tarea = this.tareas.find(t => t.id === id);
        if (tarea) {
            tarea.completada = !tarea.completada;
            this.guardar();
            this.renderizar();
            this.actualizarEstadisticas();
        }
    }

    eliminarTarea(id) {
        if (confirm('¿Estás seguro de que quieres eliminar esta tarea?')) {
            this.tareas = this.tareas.filter(t => t.id !== id);
            this.guardar();
            this.renderizar();
            this.actualizarEstadisticas();
        }
    }

    eliminarCompletadas() {
        const completadas = this.tareas.filter(t => t.completada).length;
        if (completadas === 0) {
            alert('No hay tareas completadas para eliminar');
            return;
        }

        if (confirm(`¿Eliminar ${completadas} tarea(s) completada(s)?`)) {
            this.tareas = this.tareas.filter(t => !t.completada);
            this.guardar();
            this.renderizar();
            this.actualizarEstadisticas();
        }
    }

```

```

    }

    filtrarTareas() {
        switch (this.filtro) {
            case 'pendientes':
                return this.tareas.filter(t => !t.completada);
            case 'completadas':
                return this.tareas.filter(t => t.completada);
            default:
                return this.tareas;
        }
    }

    renderizar() {
        const lista = document.getElementById('listaTareas');
        const tareasFiltradas = this.filtrarTareas();

        if (tareasFiltradas.length === 0) {
            lista.innerHTML = `
                <div class="empty-state">
                    <p>No hay tareas para mostrar</p>
                    <p>¡Agrega tu primera tarea!</p>
                </div>
            `;
            return;
        }

        lista.innerHTML = tareasFiltradas.map(tarea => `
            <li class="tarea ${tarea.completada ? 'completada' : ''}>
                <input type="checkbox" class="checkbox-tarea" ${tarea.completada ? '' : ''}>
                <span class="texto-tarea">${tarea.texto}</span>
                <button class="btn-eliminar">Eliminar</button>
            </li>
        `).join('');
    }

    actualizarEstadisticas() {
        const total = this.tareas.length;
        const completadas = this.tareas.filter(t => t.completada).length;
        const pendientes = total - completadas;

        document.getElementById('totalTareas').textContent = total;
        document.getElementById('tareasCompletadas').textContent = completadas;
        document.getElementById('tareasPendientes').textContent = pendientes;
    }

    cambiarFiltro(nuevoFiltro) {

```

```

        this.filtro = nuevoFiltro;
        this.renderizar();

        // Actualizar botones activos
        document.querySelectorAll('.controles button').forEach(btn => {
            btn.style.backgroundColor = 'white';
            btn.style.color = '#007bff';
        });
    }

    guardar() {
        localStorage.setItem('tareas', JSON.stringify(this.tareas));
    }
}

// Inicializar la aplicación
const gestor = new GestorTareas();

// Funciones globales para los botones
function mostrarTodas() {
    gestor.cambiarFiltro('todas');
}

function mostrarPendientes() {
    gestor.cambiarFiltro('pendientes');
}

function mostrarCompletadas() {
    gestor.cambiarFiltro('completadas');
}

function eliminarCompletadas() {
    gestor.eliminarCompletadas();
}
</script>
</body>
</html>

```

Ejercicio 3: Galería de Imágenes con Lightbox

Crear una galería de imágenes con efecto lightbox. Al hacer clic en una miniatura, debe abrirse la imagen en tamaño completo con navegación entre imágenes.

► Solución

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Galería con Lightbox</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: Arial, sans-serif;
      background-color: #f0f0f0;
      padding: 20px;
    }

    .galeria-container {
      max-width: 1200px;
      margin: 0 auto;
    }

    h1 {
      text-align: center;
      margin-bottom: 30px;
      color: #333;
    }

    .galeria {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
      gap: 20px;
      margin-bottom: 30px;
    }

    .miniatura {
      position: relative;
      overflow: hidden;
      border-radius: 10px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.2);
```

```
        transition: transform 0.3s ease;
        cursor: pointer;
    }

    .miniatura:hover {
        transform: scale(1.05);
    }

    .miniatura img {
        width: 100%;
        height: 200px;
        object-fit: cover;
        transition: filter 0.3s ease;
    }

    .miniatura:hover img {
        filter: brightness(1.1);
    }

    .miniatura .overlay {
        position: absolute;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        background: rgba(0,0,0,0.7);
        color: white;
        display: flex;
        align-items: center;
        justify-content: center;
        opacity: 0;
        transition: opacity 0.3s ease;
    }

    .miniatura:hover .overlay {
        opacity: 1;
    }

    /* Lightbox */
    .lightbox {
        display: none;
        position: fixed;
        top: 0;
        left: 0;
        width: 100%;
        height: 100%;
        background: rgba(0,0,0,0.9);
```

```
        z-index: 1000;
        justify-content: center;
        align-items: center;
    }

    .lightbox.activo {
        display: flex;
    }

    .lightbox-contenido {
        position: relative;
        max-width: 90%;
        max-height: 90%;
        text-align: center;
    }

    .lightbox img {
        max-width: 100%;
        max-height: 100%;
        object-fit: contain;
    }

    .lightbox-cerrar {
        position: absolute;
        top: -40px;
        right: 0;
        color: white;
        font-size: 30px;
        cursor: pointer;
        font-weight: bold;
        background: none;
        border: none;
    }

    .lightbox-cerrar:hover {
        color: #ff4444;
    }

    .lightbox-navegacion {
        position: absolute;
        top: 50%;
        transform: translateY(-50%);
        background: rgba(255,255,255,0.2);
        color: white;
        border: none;
        padding: 20px;
        font-size: 24px;
```



```

        cursor: pointer;
        transition: background 0.3s ease;
    }

    .lightbox-navegacion:hover {
        background: rgba(255,255,255,0.4);
    }

    .btn-anterior {
        left: 20px;
    }

    .btn-siguiente {
        right: 20px;
    }

    .lightbox-contador {
        color: white;
        margin-top: 10px;
        font-size: 14px;
    }
</style>
</head>
<body>
    <div class="galeria-container">
        <h1>🖼️ Galería de Imágenes</h1>

        <div class="galeria" id="galeria">
            <!-- Las imágenes se generarán dinámicamente -->
        </div>
    </div>

    <!-- Lightbox -->
    <div class="lightbox" id="lightbox">
        <button class="lightbox-navegacion btn-anterior" id="btnAnterior"><

        <div class="lightbox-contenido">
            <button class="lightbox-cerrar" id="btnCerrar">X</button>
            <img id="lightboxImagen" src="" alt="Imagen ampliada">
            <div class="lightbox-contador" id="contador"></div>
        </div>

        <button class="lightbox-navegacion btn-siguiente" id="btnSiguiente">
    </div>

    <script>
        // Array de imágenes (usando Unsplash para imágenes de ejemplo)

```

```
const imagenes = [
  {
    miniatura: 'https://images.unsplash.com/photo-1506905925346-2',
    completa: 'https://images.unsplash.com/photo-1506905925346-2',
    titulo: 'Montañas'
  },
  {
    miniatura: 'https://images.unsplash.com/photo-1507525428034-6',
    completa: 'https://images.unsplash.com/photo-1507525428034-6',
    titulo: 'Playa'
  },
  {
    miniatura: 'https://images.unsplash.com/photo-1511593358241-7',
    completa: 'https://images.unsplash.com/photo-1511593358241-7',
    titulo: 'Ciudad'
  },
  {
    miniatura: 'https://images.unsplash.com/photo-1470071459604-3',
    completa: 'https://images.unsplash.com/photo-1470071459604-3',
    titulo: 'Naturaleza'
  },
  {
    miniatura: 'https://images.unsplash.com/photo-1441974231531-c',
    completa: 'https://images.unsplash.com/photo-1441974231531-c',
    titulo: 'Bosque'
  },
  {
    miniatura: 'https://images.unsplash.com/photo-1472214103451-9',
    completa: 'https://images.unsplash.com/photo-1472214103451-9',
    titulo: 'Lago'
  }
];
```

```
let indiceActual = 0;
```

```
// Obtener referencias a elementos del DOM
```

```
const galeria = document.getElementById('galeria');
const lightbox = document.getElementById('lightbox');
const lightboxImagen = document.getElementById('lightboxImagen');
const btnCerrar = document.getElementById('btnCerrar');
const btnAnterior = document.getElementById('btnAnterior');
const btnSiguiente = document.getElementById('btnSiguiente');
const contador = document.getElementById('contador');
```

```
// Generar miniaturas
```

```
function generarGaleria() {
  imagenes.forEach((imagen, index) => {
```

```

        const div = document.createElement('div');
        div.className = 'miniatura';
        div.onclick = () => abrirLightbox(index);

        div.innerHTML = `
            
            <div class="overlay">
                <span>Ver imagen</span>
            </div>
        `;

        galeria.appendChild(div);
    });
}

// Abrir Lightbox
function abrirLightbox(index) {
    indiceActual = index;
    mostrarImagen();
    lightbox.classList.add('activo');
    document.body.style.overflow = 'hidden'; // Deshabilitar scroll
}

// Cerrar Lightbox
function cerrarLightbox() {
    lightbox.classList.remove('activo');
    document.body.style.overflow = 'auto'; // Habilitar scroll
}

// Mostrar imagen actual
function mostrarImagen() {
    const imagenActual = imagenes[indiceActual];
    lightboxImagen.src = imagenActual.completa;
    lightboxImagen.alt = imagenActual.titulo;
    contador.textContent = `${indiceActual + 1} / ${imagenes.length}`
}

// Navegación
function imagenAnterior() {
    indiceActual = (indiceActual - 1 + imagenes.length) % imagenes.length;
    mostrarImagen();
}

function imagenSiguiente() {
    indiceActual = (indiceActual + 1) % imagenes.length;
    mostrarImagen();
}

```

```

// Event listeners
btnCerrar.addEventListener('click', cerrarLightbox);
btnAnterior.addEventListener('click', imagenAnterior);
btnSiguiente.addEventListener('click', imagenSiguiente);

// Cerrar lightbox al hacer clic fuera de la imagen
lightbox.addEventListener('click', function(e) {
    if (e.target === lightbox) {
        cerrarLightbox();
    }
});

// Navegación con teclado
document.addEventListener('keydown', function(e) {
    if (!lightbox.classList.contains('activo')) return;

    switch(e.key) {
        case 'Escape':
            cerrarLightbox();
            break;
        case 'ArrowLeft':
            imagenAnterior();
            break;
        case 'ArrowRight':
            imagenSiguiente();
            break;
    }
});

// Inicializar galería
generarGaleria();
</script>
</body>
</html>

```

Conceptos practicados:

- Generación dinámica de elementos con `createElement()`
- Delegación de eventos
- Navegación con índices y operador módulo
- Event listeners para teclado
- Manipulación de clases para mostrar/ocultar elementos

- Deshabilitar scroll del body
- Array de objetos con datos
- Template strings para HTML dinámico